

Design Reflection: Calculator App

Vu Duy Quang

Student ID: 202417186

Global ICT - Hanoi University of Science and Technology

Course: IT3100E - Object-Oriented Programming

Reflection

In this homework, I implemented a calculator application following the Model–View–Controller (MVC) design pattern. The main design decision was to separate the calculation logic from the graphical user interface. The `CalculatorModel` is responsible for storing the internal state and handling all calculation rules. The `CalculatorView` only creates and displays the Swing user interface, while the `CalculatorController` connects user actions from buttons to the corresponding methods in the model.

The calculator supports basic arithmetic operations including addition, subtraction, multiplication, division, and modulo. It also supports clear, delete, chained operations such as $12 + 3 + 4 =$, replacing an operator when the user presses two operators consecutively, and error handling for division or modulo by zero. These features make the calculator behave more like a real basic calculator rather than only evaluating one isolated operation.

The most important implementation idea was treating the model as a state machine. The model keeps several fields such as `currentInput`, `storedValue`, `operator`, `waitingForSecondOperand`, `justCalculated`, and `error`. Each user action, such as pressing a digit, an operator, or the equals button, causes a transition from one state to another. This made the logic clearer and easier to test.

A strong point of my implementation is that the model does not depend on Swing classes. Therefore, I could test the calculation logic separately using a test class before connecting it to the GUI. This improves maintainability and makes the design cleaner. Another strength is that edge cases were considered explicitly instead of being left undefined. Overall, this project helped me understand how MVC can be used to organize an interactive Java application with clear responsibilities among classes.